

Christopher Strachey

Towards a Formal Semantics and WG 2.2

Peter Mosses, IFIP WG 2.2 Meeting, Brussels, Belgium, 15–19 September 2026

Christopher Strachey

Sources

Biography (1985)

Christopher Strachey, 1916–1975
A Biographical Note
MARTIN CAMPBELL-KELLY

Foreword

Christopher Strachey was one of the most original computer scientists of his generation: his fields were computer design and programming, and he made important contributions to them both. He liked to say (not quite accurately) that he had changed the direction of his career every seven years. The different phases of his life and career are reflected in the five sections of this note.

- 1. Family Background and Education (1916–1939)
- 2. War Service and Schoolmastering (1939–1952)
- 3. National Research and Development Corporation (NRDC) (1952–1959)
- 4. Private Consultancy and Cambridge University (1959–1965)
- 5. Oxford University (1965–1975)

Born in 1916 into one of England’s more prominent families, Christopher Strachey was educated at Gresham’s School, Norfolk, and at King’s College, Cambridge. He spent the war years in radar research, after which he spent several years as a schoolmaster. His career in computing did not effectively begin until 1951 when he started to program the machines at the National Physical Laboratory and Manchester University, while still a master at Harrow School. His ability as a programmer rapidly came to the attention



Christopher Strachey, 1916–1975.

of Lord Halsbury of NRDC, who appointed him technical officer in 1952. At NRDC, Strachey was involved in many projects, but particularly in the logical design of computers. In 1959 he left NRDC to set up in practice as a private consultant in computer design and applications. In 1962 he also began work in the University Mathematical Laboratory, Cambridge, where, in the nominal position “part-time research assistant,” he worked on the CPL programming language. In 1965, he wound up his consultancy and resigned from Cambridge in order to form the Programming Research Group at Oxford University. Under his leadership the group quickly established an international reputation in the theory of programming. At Oxford, in collaboration with Dana Scott, he produced the work of which he was most proud, the foundation of denotational semantics; this work was just beginning to bear fruit when he died in 1975.

This biographical note would not have been possible in its present form had Strachey not been a lifelong hoarder of his personal papers. After his death, his sister, Barbara Halpern, allowed the Contemporary Scientific Archives Centre at Oxford to bring his pa-

© 1985 by the American Federation of Information Processing Societies, Inc. Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the AFIPS copyright notice and the title of the publication and its date appear, and notice is given that the copying is by permission of the American Federation of Information Processing Societies, Inc. To copy otherwise, or to republish, requires specific permission.
Author’s Address: Department of Computer Science, University of Warwick, Coventry CV4 7AL, England.
Categories and Subject Descriptors: A.0 [General]—biographies, C. Strachey; K.2 [History of Computing]—people, C. Strachey, software, systems. General Terms: Design. Additional Key Words and Phrases: CPL, denotational semantics, NRDC, Pegasus.
© 1985 AFIPS 0164-1239/85/010019-04\$01.00/00

HOSC Special Issue (2000)

HIGHER-ORDER AND
SYMBOLIC
COMPUTATION

Volume 13, Nos. 1/2, May 2000

A Special Issue Dedicated to Christopher Strachey		
Editorial	Olivier Danvy and Carolyn Talcott	5
A Foreword to ‘Fundamental Concepts in Programming Languages’	Peter D. Mosses	7
Fundamental Concepts in Programming Languages	Christopher Strachey	11
Christopher Strachey—Understanding Programming Languages	Rod Burstall	51
From Macrogeneration to Syntactic Abstraction	R. Kent Dybvig	57
Christopher Strachey: Recollections of His Influence	Mike Gordon	65
Cambridge and CPL in the 1960s	David Hartley	69
A Hard Act to Follow	C.A.R. Hoare	71
Christopher Strachey: A Personal Recollection	Michael Jackson	73
My Years with Strachey	Peter J. Landin	75
From Language Concepts to Implementation Concepts	Robert Milne	77
Reminiscences of Christopher Strachey	Roger Penrose	83
Christopher Strachey and the Cambridge CPL Compiler	Martin Richards	85
Induction, Domains, Calculi: Strachey’s Contributions to Programming-Language Engineering	David A. Schmidt	89
Some Reflections on Strachey and His Work	Dana Scott	103
Christopher Strachey and Fundamental Concepts	Joe Stoy	115
Abstract Models of Storage	Robert D. Tennent and Dan R. Ghica	119
Continuations Revisited	Christopher P. Wadsworth	131
Continuations: A Mathematical Semantics for Handling Full Jumps	Christopher Strachey and Christopher P. Wadsworth	135

Strachey 100 (2016)

STRACHNEY 100



Celebrating the life and research of
Christopher Strachey

University of Oxford
Department of Computer Science

18th–19th November 2016

Education and first employment

1935–39: King's College, Cambridge University

- Maths, then switched to Physics
- Natural Sciences tripos



1939–45: STC (Standard Telephones and Cables)

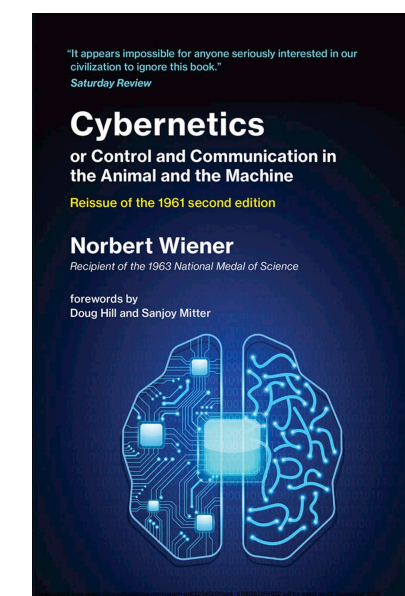
- research physicist
- radar valve design
- use of differential analyser



Schoolmaster, machinecode programmer

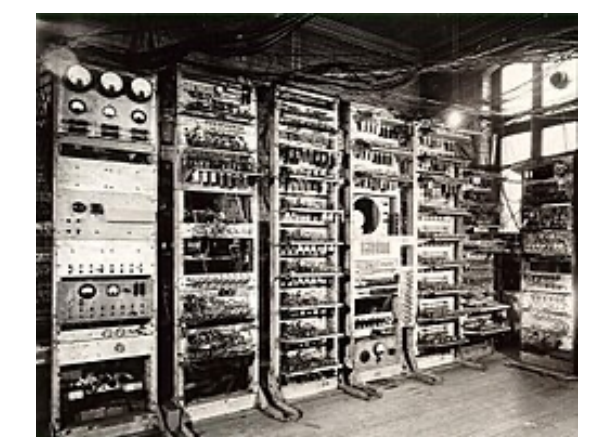
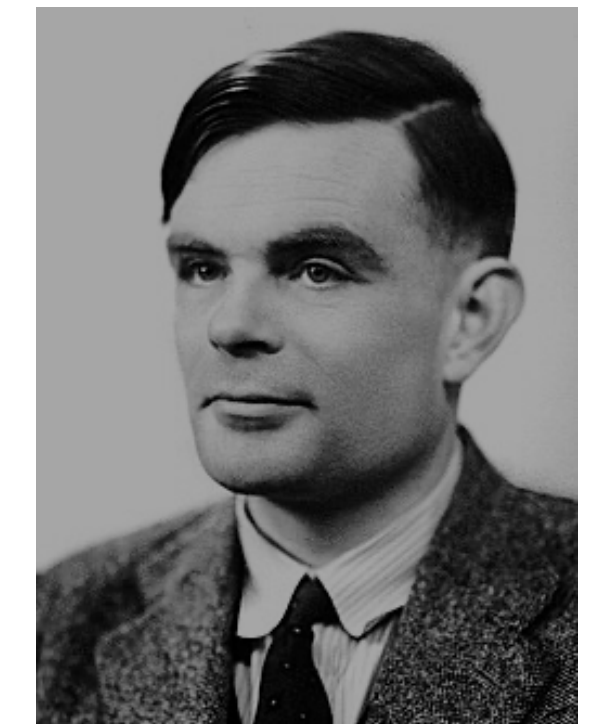
1945–48: St. Edmunds

- maths and physics



1948–51: Harrow

- bought Wiener's (1948) Cybernetics book
- tried to implement a draughts program on the Pilot ACE at NPL
- obtained the manual for the Ferranti Mark I from Alan Turing
- implemented a simulation, ending with the National Anthem!



Technical officer, NRDC

1952–53: Toronto and USA

- paper on draughts program (*ACM National Conference*, Toronto)
- St. Lawrence Seaway simulation on *Ferranti Mark I* (*FERUT*, Toronto)
- study of instruction sets (US research labs and manufacturers)



1953–59: London

- evaluation and code redesign of *Elliott 401*, logical design of *Pegasus*
- matrix algebra and aircraft flutter problem (assistant: Roger Penrose)
- multiprogramming/time-sharing patent, paper



Private consultant

1959–65: London

- public debate about programming languages
- paper on improving efficiency of *Algol 60* (with Maurice Wilkes, 1959)
- various computer design and applications projects
- autocode for the *Ferranti Orion*
- employed Peter Landin as an assistant (1960–64)
- encouraged Landin's theoretical work on semantics



Mathematical Laboratory, Cambridge

1962–65: *CPL* (Combined Programming Language)

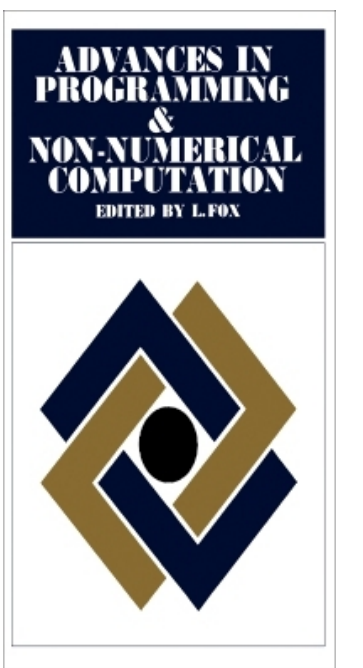
- ▶ Senior Research Fellow, Churchill College
- ▶ part-time work on *CPL* and compiler for Cambridge *Titan* computer
- ▶ invited by Wilkes to design and implement *CPL*
 - with David Barron and David Hartley, also Eric Nixon and John Buxton
- ▶ paper: *The Main Features of CPL* (with Barron et al., 1963)
- ▶ reference manual: *CPL Working Papers* (ed. Strachey, 1966)



Private consultant

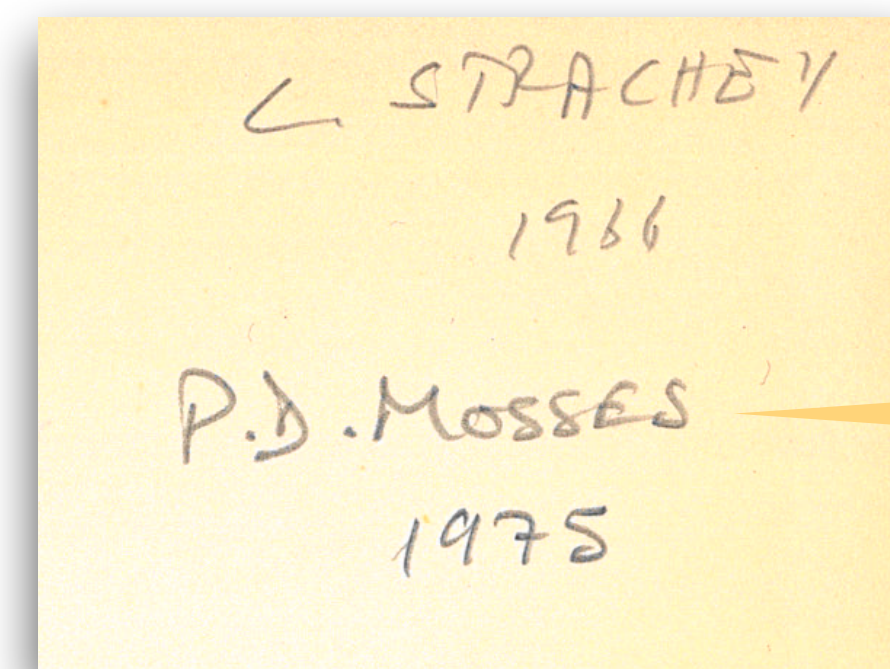
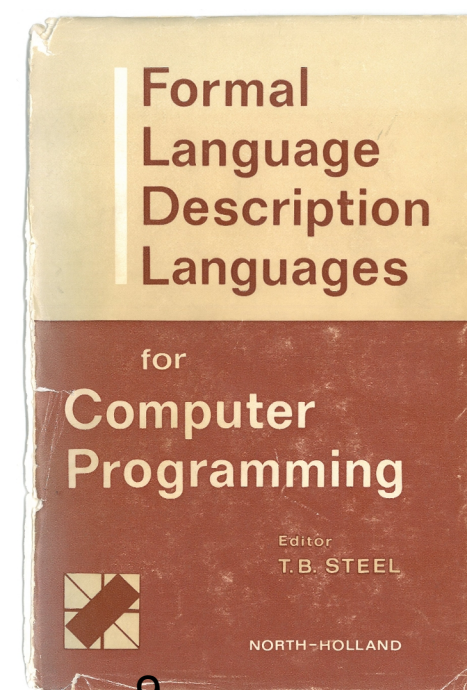
1963: Summer school (with Fox et al., Oxford University)

- lectures on *Programming* (with David Barron)
- book: *Advances in Programming and Non-Numerical Computation* (1966)



1964: *Towards a Formal Semantics*

- *IFIP TC2 Working Conference on Formal Language Description Languages*
 - 1966: proceedings published



JENS PALSBERG
2024

Christopher Strachey

Towards a Formal Semantics

Towards a formal semantics

– background

1962–64:

- ▶ Strachey *needed formal semantics* for *precise description* of *CPL*; he also wanted to isolate *fundamental concepts* of programming languages
- ▶ Penrose had tried to persuade Strachey to use the λ -calculus around 1958; Strachey later borrowed Church's book on the λ -calculus from Penrose
- ▶ Landin's approach to a semantics for *Algol 60* was based on an extension of the λ -calculus with *side-effects* and *jumps*, and an *abstract machine*

Towards a Formal Semantics

1. INTRODUCTION

The end product of this process would be a description of the programming language in terms of a precisely, but probably not formally, specified set of primitive ideas and notations; the first task of this paper is an attempt to ascertain and identify these primitive ideas. Once the basic ideas have been isolated, it becomes possible to describe the main elements of the programming language in terms of them; the second task of this paper is an attempt to do this in an informal manner.

Towards a Formal Semantics

2. BASIC CONCEPTS

It seems that the imperative features of a programming language introduce some basically new ideas not easily incorporated into a descriptive system. In Landin's paper here on the semantics of ALGOL, he gets over this problem by an alteration of his evaluating mechanism, which now becomes an essential feature of the description of ALGOL.

It is the aim of this paper to show that with the introduction of a few new basic concepts it is possible to describe even the imperative parts of a programming language in terms of applicative expressions, including even such "essentially computer-oriented" ideas as assignment and transfers of control. As a result, it is no longer an essential part of the basic semantic description of the language to postulate an "evaluating mechanism" or notional computer, however abstract.

Towards a Formal Semantics

3. ASSIGNMENT COMMANDS

- general form: $\epsilon_1 := \epsilon_2$ where ϵ_1 and ϵ_2 are *expressions*

4. L-VALUES AND R-VALUES

- an **L-value** α is a generalised *address* – an area of a *store* σ
- an **R-value** β is a *bit-string* – representing the *value stored* at an L-value
- the operator $C(\alpha)$ “loads” the value stored at α : $\beta = C(\alpha)(\sigma)$;
the operator $U(\alpha)$ “updates” the value stored at α : $\sigma' = U(\alpha)(\beta', \sigma)$

Towards a Formal Semantics

5. DIGRESSION ON NOTATION

- **application:** $E_1 E_2$
- **λ -expressions:** $\lambda x . E$ or $\lambda(x, y) . E$
- **lists:** (E_1, E_2)
- **composition** (“functional product”): $E_1 \cdot E_2 = \lambda x . (E_1 (E_2 x))$
- **currying:** $\chi = \lambda f . \lambda x . \lambda y . f(x, y)$
- **uncurrying:** $\nu = \lambda g . \lambda(x, y) . g x y$

Towards a Formal Semantics

6. EVALUATION OF EXPRESSIONS

- **L-value** of ϵ_1 : $\alpha = L(\epsilon_1, \sigma) = \chi L \epsilon_1 \sigma$
- **R-value** of ϵ_2 : $\beta = R(\epsilon_2, \sigma) = \chi R \epsilon_2 \sigma$

Towards a Formal Semantics

7. CONDITIONAL EXPRESSIONS

- syntax: $\epsilon_0 = \{\epsilon_1 \rightarrow \epsilon_2, \epsilon_3\}$
- basic **selector** function “*if*” :
 - (*if* β (x, y)) is x when β represents *true*, and y when β represents *false*
- “We might expect to write $L(\epsilon_0, \sigma) = \text{if}(R(\epsilon_1, \sigma))(L(\epsilon_2, \sigma), L(\epsilon_3, \sigma))$
but this would involve evaluating both arms of the condition.”
- “An alternative [...] : $L(\epsilon_0, \sigma) = \text{if}(\chi R \epsilon_1 \sigma) (\chi L \epsilon_2, \chi L \epsilon_3) \sigma .$ ”

Towards a Formal Semantics

8. COMMANDS

- assignment:

$$\text{Prog } \{\epsilon_1 := \epsilon_2\} = \lambda\sigma. U(L(\epsilon_1, \sigma)) (R(\epsilon_2, \sigma), \sigma) \qquad \sigma' = U(\alpha)(\beta', \sigma)$$

- conditional commands:

$$\begin{aligned} \text{Prog } \{\mathbf{test } \epsilon_1 \mathbf{ then do } \gamma_1 \mathbf{ or do } \gamma_2\} = \\ \lambda\sigma. \text{if } (R(\epsilon_1, \sigma)) (\text{Prog } \gamma_1, \text{Prog } \gamma_2) \sigma \end{aligned}$$

Towards a Formal Semantics

9. SIDE EFFECTS

- *L*-values: $L'(\epsilon_1, \sigma) = \alpha, \sigma'$
- *R*-values: $R'(\epsilon_2, \sigma') = \beta, \sigma''$
- *Prog* $\{\epsilon_1 := \epsilon_2\} = \lambda\sigma. (\lambda(\alpha, \sigma'). U \alpha (R'(\epsilon_2, \sigma')))(L'(\epsilon_1, \sigma))$

Towards a Formal Semantics

10. BLOCKS, DECLARATIONS, ETC.

- based on Landin's translation of *Algol 60* blocks to applicative expressions
 - the block body is translated to the body of a λ -abstraction
 - variables declared at the head of the block are translated to bound variables
 - the λ -abstraction is applied to the initial values of the declared variables

*Strachey did not use **explicit** environments to map names to values until 1970*

Towards a Formal Semantics

11. SEQUENCING, JUMPS, LOOPS, ETC.

- examples of how composition of the semantics of commands is affected by jumps and conditions
- loops involve applying the “fixed point” operator Y to abstractions of command semantics

No known functional model for Y until 1969

Strachey did not use continuations until 1974

Towards a Formal Semantics

12. COMMENTS AND CONCLUSION

- “the analysis given above is both informal and incomplete”
 - developed together with the CPL language and its compiler
 - lack of space and time

Towards a Formal Semantics

DISCUSSION

Insofar as possible, pertinent discussion has been reproduced verbatim, omitting only extraneous noise. Selection of material from the discussions has been done by the Editor; the participants have not had an opportunity to amend their remarks. Thus, what is reported is what was actually said, not what the speaker might have wished he had said.

Programming Research Group, Oxford

1965: Funding for PRG at Oxford University Computing Laboratory

- SRC research grant award (to Fox)

1965–66: Strachey visits MIT (7 months)

- graduate course: *Mathematical Theory of Programming Languages*

1966: Strachey starts as leader of the PRG

- Reader, and Fellow of Wolfson College
- joined by David Park (until 1968)

Programming Research Group, Oxford

The Strachey's Philosophy

The strongly held common philosophy of the Department (previously Oxford University Computing Laboratory) is admirably expressed in the following quotation from Christopher Strachey, the founder of the Programming Research Group which now forms part of Computing Science at the Department:

"It has long been my personal view that the separation of practical and theoretical work is artificial and injurious. Much of the practical work done in computing, both in software and in hardware design, is unsound and clumsy because the people who do it have not any clear understanding of the fundamental design principles of their work. Most of the abstract mathematical and theoretical work is sterile because it has no point of contact with real computing. One of the central aims of the Programming Research Group as a teaching and research group has been to set up an atmosphere in which this separation cannot happen."

See also maths.ox.ac.uk/node/41355

Christopher Strachey and

IFIP WG 2.2

*Programming Language Description /
Formal Language Description Languages /
Formal Description of Programming Concepts*

Establishment of WG 2.2

January 1967

Invitation from T. B. Steel, Jr. (excerpt):

Accordingly, it has been proposed, and the proposal endorsed by the appropriate IFIP governing bodies, that IFIP TC-2, as the proper organization to institute such work, undertake the establishment of a Working Group on Programming Language Description (WG2.2) whose charter is the investigation, evaluation, and development of methodologies, both formal and informal, for the description, both syntactic and semantic, of programming languages. The activities of this Working Group will include conducting a survey of existing programming language description methods, the establishment of criteria for selecting description methods and other studies as it shall from time to time initiate, subject to the approval of TC-2 as its parent committee.

Initial meeting of WG 2.2

Sardinia, September 1967

Minutes of the first meeting of WG 2.2 on Formal Language Description Languages

3. Presentations

3.1 C.Strachey presented a number of concepts which he claims to be fundamental for programming languages. He stated that higher level concepts can intuitively be formed without, from the start, basing them as simpler ones and that these concepts then can be sharpened when proceeding to more detailed understanding. The concepts discussed were pairs of properties:

- a) lefthand-values (the results of the evaluation of the lefthand side of assignment statements) versus righthand-values (the results of the evaluation of expressions). A lefthand-value must allow the transition to a righthand-value (address/value relationship).
- b) manifest versus latent properties in a program; manifest roughly means detectable at compile time, latent detectable only at run time.
- c) constant versus variable lefthand values (constant values cannot be changed).
- d) fixed versus free righthand values (example: a fixed function is insensitive to other functions' side-effects).
- e) scope versus extent of variables; scope concerns the program regions where a variable is known, extent concerns the points where the denotation of a variable name is established and erased.

Van Wijngaarden strongly disagreed that the mentioned concepts have general importance. There followed a general discussion about the relationship address/value and the general relationship name/object.

Second WG 2.2 meeting

Copenhagen, July 1968

Minutes of the second meeting of WG 2.2 on
Formal Language Description Languages

Hotel Marina, Vedbaek-Copenhagen, Denmark,
22-28 July 1968

- 2.2 There was disagreement on whether for the objectives of the working group the better approach is the application of a description method to existing programming languages (supported by Caracciolo, Paul), or to a variety of smaller, possibly idealized, examples (supported by Gorn, Strachey). In the latter approach emphasis would be on the separate description of separable language features. A crucial question is whether it will be possible to combine the descriptions (or "local models") of individual language features to a consistent overall language description.

Second WG 2.2 meeting

Copenhagen, July 1968

2.3 Ingerman demanded the setting up of a certain frame-work before entering the discussion of the various description methods. He proposed the following:

- (1) Occam's razor applies
- (2) we need a machine
- (3) we need a machine description
- (4) there must exist some kind of language
- (5) mapping in (3) of (4) into (2)

Gorn added another point:

- (4.5) there exists a user.

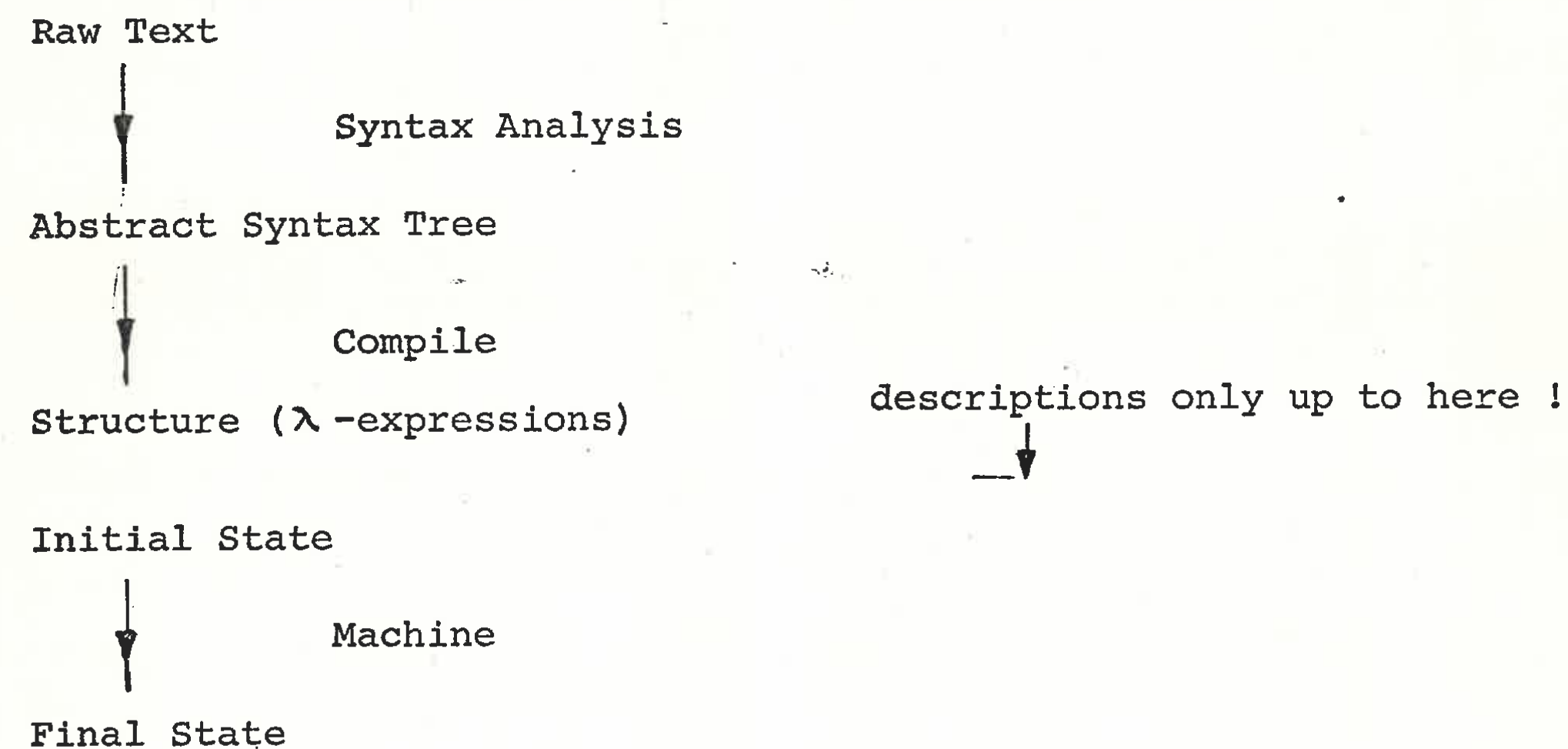
Strachey objected to this frame-work as it presupposes the need for a machine, excluding description techniques following entirely the translation approach. De Bakker claimed that the axiomatic approach would not fit into the frame-work.

Second WG 2.2 meeting

Copenhagen, July 1968

8. Presentation by C. Strachey on his views

No machine is used in the language description. The proposed scheme is the following:



This scheme puts everything in the translation process. The machine interpreting λ -expressions, being a standard machine, needs not be part of a language description.

It is important to demonstrate what can be done at compile time (i.e., what is "manifest"), and what not. This corresponds to the Compile step in the above scheme.

The translation is achieved by three sets of mutually recursive transformation rules, namely
transformation rules for left-hand values,
transformation rules for right-hand values,
transformation rules for commands.

A paper on Fundamental Concepts in Programming Languages, given at the 1967 NATO Summer School in Copenhagen, will be made available to the members of WG 2.2

Discussion: Caracciolo (supported by Paul) said that Strachey's scheme relies on the equivalence notion of the λ -calculus, and that there are, however, several different conceivable notions of equivalence. Introducing equivalence relations is important as they define what, in a description, is to be disregarded.

Boehm asked what the primitives are, i.e. things not defined in terms of λ -expressions.

2. Papers distributed at the Vedbaek meeting

/24/ STRACHEY, C.: Some Features of Programming Languages.
25 July 1968.

Second WG 2.2 meeting

Copenhagen, July 1968

13. Presentation by C.C. Elgot on Iteration and Recursion

Reference: Vedbaek 20.

13.1 Iterative processes, usually considered in the theory of machines, can also be used to characterize recursive partial functions.

Partial functions

$$f : N^r \rightarrow N^s$$

are considered, where N is the monoid of non-negative integers, and N^r is the r -fold product $N \times \dots \times N$. A class of such functions is called distinguished if it contains certain specific partial functions (induced functions, unit, successor, antiunit, predecessor) and is closed under certain operations (composition, cylindrification, union, iteration). The smallest distinguished class coincides with the class of recursive partial functions.

Discussion: Strachey observed a close relation between the operator $*$ and the Y -operator. Elgot said that a formulation in combinatory logic would be a step backward. Boehm stated that he had a system similar to Elgot's.

14. Presentation by C. Strachey on the Operator Y

The operator Y is used for the explication of loops. The application of the operator P on the program text gives the corresponding λ -expression, which for a loop-statement gives

$$P \{ \text{while } \epsilon \text{ do } \gamma \} = Y \{ \lambda \theta. \text{Cond}(\theta. P \gamma, I) : \mathcal{R} \epsilon \}$$

where Y determines the fix-point of a function:

$$\text{if } YF = g, \text{ then } Fg = g,$$

the function Cond is defined as

$$\text{Cond}(p, q) \beta = \begin{cases} p & \text{if } \beta \text{ is true} \\ q & \text{if } \beta \text{ is false} \end{cases}$$

$\mathcal{R}$ defines the text transformation on ϵ such that the application of $\mathcal{R} \epsilon$ to the state σ gives the evaluation of ϵ (it is assumed here that the evaluation of ϵ has no side-effects), and the operator $:$ is defined as

$$(f : g) \sigma = f(g \sigma) \sigma$$

Third WG 2.2 meeting

Vienna, April 1969

Minutes of the third meeting of WG 2.2 on
Formal Language Description Languages

Hotel Stefanie, Vienna, Austria
21-25 April 1969

Welcome by H. Zemanek

H. Zemanek welcomed the working group to Vienna. He outlined several of IFIP's current plans and problems, mentioning in particular the widening of the scope of TC 2 in generalizing from "programming languages" to "programming". WG 2.2 is invited to make a similar step.

Third WG 2.2 meeting

Vienna, April 1969

5. Presentation by C. Strachey on the Isolation of Features in Programming Languages

Reference: C. Strachey: Fundamental Concepts in Programming,
WG 2.2 Bulletin No. 1

A number of concepts are isolated:

- 1 Types (see also 3.2)
 - Class I: all types of expressions are manifest, type information is used for determining polymorphic operators, type information is suppressed at run time
 - Class II: types are latent, determinable at run-time only, type information is used to determine polymorphic operators, type information must be carried along
 - Class III: no types, no polymorphic operators
 - a) no access to addressing structure (LISP)
 - b) access to addressing structure assembly language)
- (2) First_and_second_class_objects
 - first class objects: can be values of variables and expressions (reals, etc.)
 - second class objects: cannot be values of variables and expressions (e.g. procedures in ALGOL 60)

- (3) Left_and_right-hand_values (see also the Alghero minutes)

There is, correspondingly, an L-evaluation and an R-evaluation.

Questions of interest:

- sharing, non sharing
- are there expressions (other than names_ producing L-values ?
- are there programmer-defined functions producing L-values ?
- can the L-value associated with a name be changed ?

- 4 Extent

The extent of a quantity is the running period in which the quantity exists (to be differentiated from scope). Has bearing on garbage collection.

- (5) Definitional_structure

functional abstraction (definition of functions and subroutines_
possible ways of parameter passing
scope of names
side-effects

Fundamental Concepts in Programming Languages



Higher-Order and Symbolic Computation, 13, 11–49, 2000
© 2000 Kluwer Academic Publishers. Manufactured in The Netherlands.

Fundamental Concepts in Programming Languages

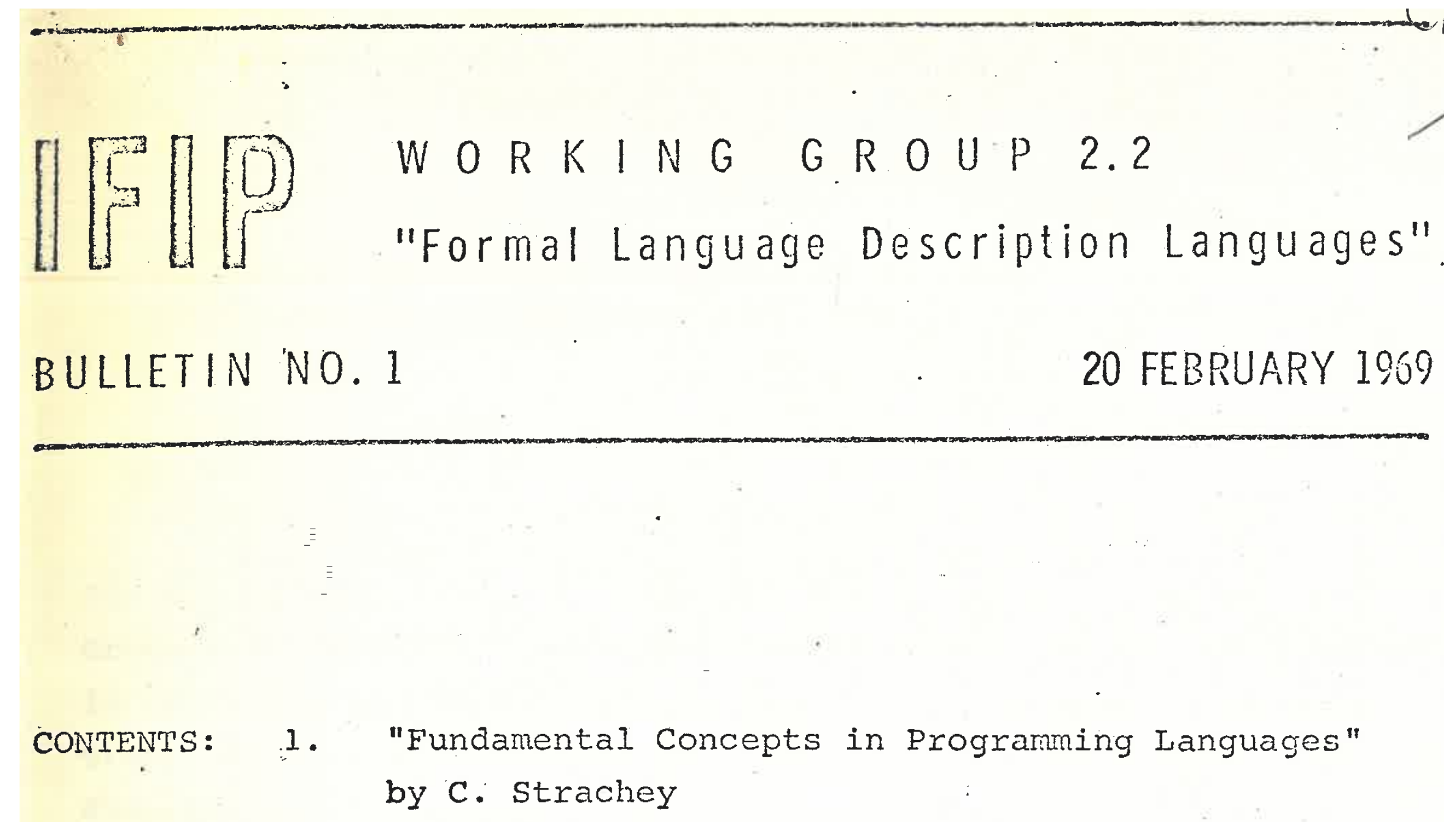
CHRISTOPHER STRACHEY

Reader in Computation at Oxford University, Programming Research Group, 45 Banbury Road, Oxford, UK

Abstract. This paper forms the substance of a course of lectures given at the International Summer School in Computer Programming at Copenhagen in August, 1967. The lectures were originally given from notes and the paper was written after the course was finished. In spite of this, and only partly because of the shortage of time, the paper still retains many of the shortcomings of a lecture course. The chief of these are an uncertainty of aim—it is never quite clear what sort of audience there will be for such lectures—and an associated switching from formal to informal modes of presentation which may well be less acceptable in print than it is natural in the lecture room. For these (and other) faults, I apologise to the reader.

There are numerous references throughout the course to CPL [1–3]. This is a programming language which has been under development since 1962 at Cambridge and London and Oxford. It has served as a vehicle for research into both programming languages and the design of compilers. Partial implementations exist at Cambridge and London. The language is still evolving so that there is no definitive manual available yet. We hope to reach another resting point in its evolution quite soon and to produce a compiler and reference manuals for this version. The compiler will probably be written in such a way that it is relatively easy to transfer it to another machine, and in the first instance we hope to establish it on three or four machines more or less at the same time.

The lack of a precise formulation for CPL should not cause much difficulty in this course, as we are primarily concerned with the ideas and concepts involved rather than with their precise representation in a programming language.



Third WG 2.2 meeting

Vienna, April 1969

9. Report by C. Strachey on Discussions of Language Concepts

The subgroup has discussed the treatment of types (class I and class II languages, but not class III), sharing, first and second class objects, L- and R-values, notions of equivalence.

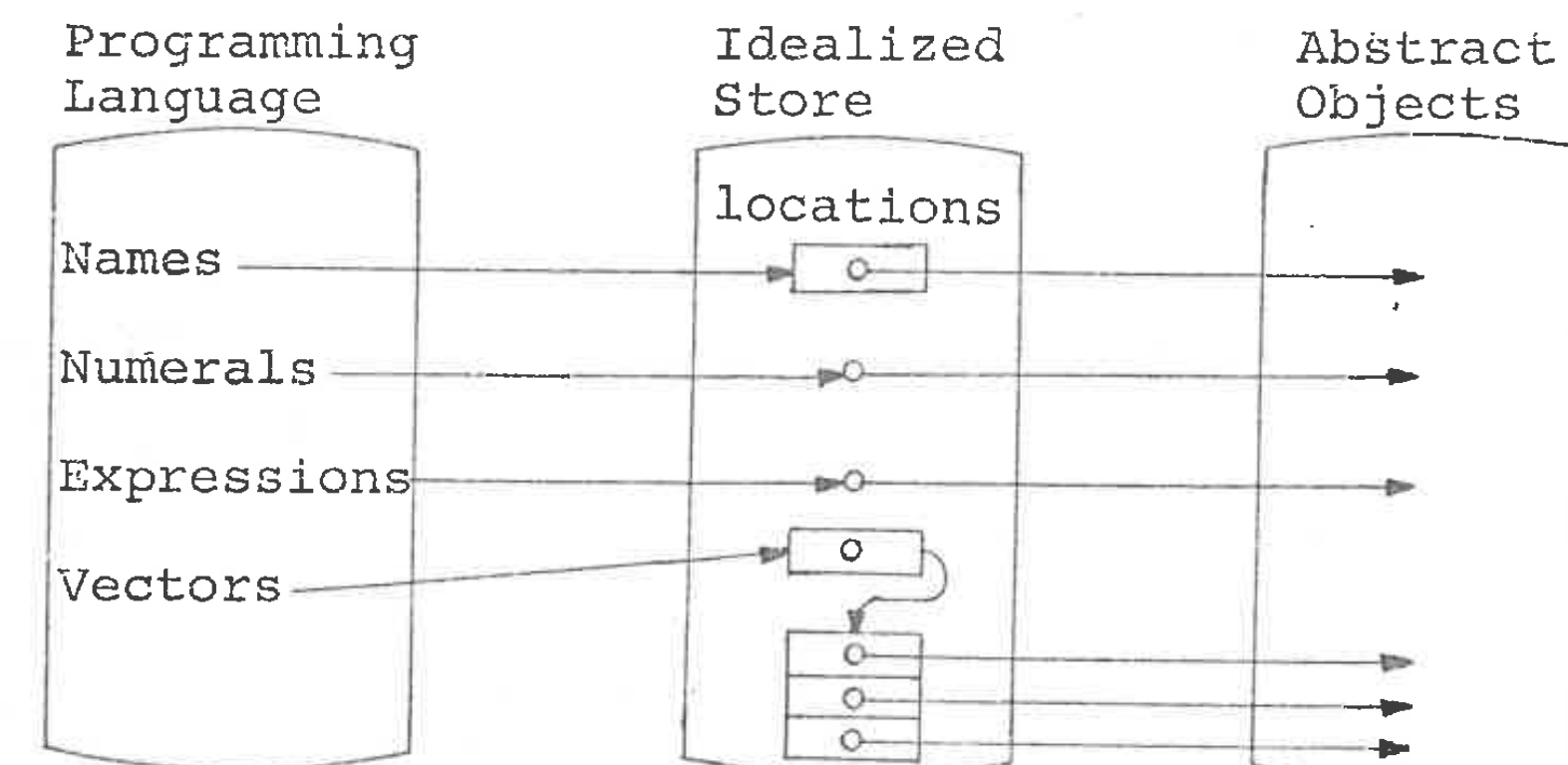
Discussion:

Caracciolo, Scott: We should discuss in more formal terms. Kernel questions concern things which distinguish higher level languages from assembly languages, i.e. variables, procedures, parameters. The difference between L- and R-values in Strachey's sense and the ALGOL 68 reference mechanism is still open.

Strachey: We are not in the position to make discussions completely formal.

10. Presentation by C. Strachey on Assignment and Reference

A conceptual model is presented, illustrated by the following diagram (later on referred to as the 3-sausage diagram):



Names and vectors) have R-values and L-values (locations). Locations are atomic or non-atomic. The store specifies the relationship between L-values and R-values:

$$\sigma \alpha = \beta$$

σ a store, α an L-value, β an R-value)
 σ can be written as a conditional expression.

Third WG 2.2 meeting

Vienna, April 1969

17. Administrative Matters

It was suggested for future meetings to have a pre-defined agenda with well-prepared presentations for the first day. Presentations could be aided by written material in the Bulletin.

The next meeting will be in England in September. Other suggestions for meeting places were Sicily (Caracciolo) and Finland (Kurki-Suonio).

The next after the next meeting is tentatively planned to be at the US East Coast in August 1970.

22. D. Scott: An Outline of Topics in the Subject Matter of WG 2.2

Five categories are identified, listed in the following together with a pertinent crucial question:

SUBSTANCE

Is there something to discuss independent of language ?

SEMANTICS

Is there a definition of computation ?

DEDUCTION

Can the result of understanding programs be formalized ?

DESCRIPTION

Can the systems be communicated ?

DESIGN

Can the languages be improved ?

Fourth WG 2.2 meeting

Essex, September 1969

Minutes of the fourth meeting of WG 2.2 on
Formal Language Description Languages

Essex University, Colchester, Essex, England
8-12 September 1969

Members present at the meeting:

C. Boehm, K. Čulik, J.W. de Bakker, J.J. Florentin, J.V. Garwick,
R. Kurki-Suonio, P.J. Landin, P. Lucas, J. McCarthy, M. Nivat,
M. Paul, D. Scott, T.B. Steel, C. Strachey, A. van Wijngaarden,
K. Walk.

Observers:

H. Bekić, M. Bell, K.E. Blum^x), J.W. Carr III^x), C.A.R. Hoare
(the first two days), J.G. Laski^x), P.E. Lauer, T. Maeno, E. Nixon^x),
D. Park, M. Richards.

- 3. Presentation by D. Scott on the Methods of de Bakker and Hoare,
and on questions of equivalence of programs
- 4. Discussion on the Purpose and Preceeding of the Working Group
- 5. Presentation by D. Scott on Proofs about Programs
(A Discussion of Floyd's and Hoare's Work)
- 6. Presentation by J. McCarthy on "Blobs" and on Proving the
Correctness of Compilers
- 7. Presentation by K.E. Blum on the Possibility of a Uniform Method
for Describing Semantics

- 8. Discussion on How to Proceed
- 9. Presentation by A. van Wijngaarden on Collaterality and
Parallelism in ALGOL 68
- 10. Presentation by P. Lucas on Collateralism and Parallelism in PL/I
- 11. Presentation by J. McCarthy on "Parallel"
- 12. Presentation by K. Culik on Structural Equivalence of
Simple Assembly Language Programs
- 13. Discussion on the Next Meeting
- 14. Discussion on Data Structures
- 15. Presentation by H.Bekić on Manna's Work
- 16. Discussion about keeping grand state vector versus recursive
formulation
- 17. Presentation by D.Scott on Fix-Points
- 18. Presentation by D.Scott - Continued

Scott–Strachey collaboration

1969:

- Dana Scott met Strachey at the third WG 2.2 meeting (Vienna), and extended his sabbatical to visit Strachey; they both attended also the fourth meeting (Essex)
- Scott developed ***domain theory*** in seminars at Oxford

1970:

- paper: *Outline of a **Mathematical Theory** of Computation* (by Scott)

1971:

- paper: *Towards a **Mathematical Semantics** for Computer Languages* (with Scott)

Fifth WG 2.2 meeting

Postponement

February 18, 1970

TO: IFIP WG2.2

FROM: Chairman WG2.2

As you know WG2.2 had planned a meeting in the Eastern United States for the week of 1970 April 13-17. Most of you do not know that instead of Princeton, the meeting had been rearranged for New Haven, Connecticut since Ned Irons, who was asked to assist, had moved to Yale University.

Since that time several things have happened, including the fact that Dana Scott and Chris Strachey have done some work that bears significantly on WG2.2's mission. This work should be reported prior to WG2.2 spending time that might be wasted and it will not be possible for this to be done during the scheduled week. For this, and other, reasons, it is necessary to postpone the WG2.2 meeting. It is the likelihood of this being the case that caused me to delay formal invitations. I know some of you were concerned about the shortness of time.

Fifth WG 2.2 meeting

Boston, September 1970

Minutes of the fifth meeting of WG 2.2 on
Formal Language Description Languages

Statler Hilton Hotel, Boston, USA
8-11 September 1970

Members present at the meeting:

Chairman acting: D. Scott

K.E. Blum, J.W. de Bakker, C.C. Elgot, J.J. Florentin. J.V. Garwick,
V.M. Kurochkin, P.J. Landin, P. Lucas, J. McCarthy, E. Nixon,
R. Tabory, A. van Wijngaarden, K. Walk.

Observers:

W. Bibel, S. Igarashi, C.B. Jones, R. Rustin, J. Sammet (Thursday,
Friday), P. Wegner (Friday).

2. Opening Discussion

Presentations that could be given during the meeting were identi-
fied and scheduled (Scott, Igarashi, Lucas-Jones, Walk).

There was discussion on a possible contribution of WG 2.2 to the
IFIP Congress in August 71 and about a separate working conference
on the subject of WG 2.2. Tabory suggested specifically a state of
the art presentation at the IFIP Congress. Walk was charged with
finding out the possibilities.

Scott suggested to consider new blood for WG 2.2 and to reconsider
membership of those who never have attended a meeting. (The subject
was taken up again on Friday.)

There was also discussion on the scope of WG 2.2. Walk read the
original invitation letter to the group, which contains a statement
about the goals and the scope of the working group. de Bakker ob-
served that a survey and comparison of description methods was done
only in the first two meetings, and that the scope in the meanwhile
has broadened to mathematical theory of computation. No new de-
scription methods have shown up during the last two years.

- 3. Discussion about Teaching Computer Science
- 4. Presentation by D. Scott on Mathematical Semantics, I.
- 5. Presentation by D. Scott on Mathematical Semantics II.
- 6. Presentation by C.B. Jones on Proving Correctness of
Implementation Techniques from Formal Definition
- 7. Presentation by S. Igarashi on the Correctness of a
Conceptual Compiler
- 8. Remark by D. Scott on Blocks and Procedures
- 9. Presentation by K. Walk on a General Storage Model
- 10. Presentation by P. Landin on an Algebraic Approach
- 11. Presentation by C.C. Elgot on Algebraic Theories
- 12. Discussion on the Meeting
- 13. Discussion on the Scope of WG 2.2, and Future Meetings

13.4 Title of WG 2.2

The majority of members present felt unsatisfied with the present title.

Three titles were finally proposed, and a straw vote taken on the
question: "who cannot live with that title ?"

The result was:

Formal Language Description	7 yes
Formal Aspects of Progr. Languages	6 yes
Theory of Programming Languages	0 yes

Christopher Strachey
(concluded)

Programming Research Group, Oxford

1971:

- Strachey appointed *Professor* at Oxford University

1972:

- SRC funding ends, PRG incorporated in Oxford University
- paper: *Varieties of Programming Language*
- paper: *OS6—An Experimental Operating System for a Small Computer*
(in two parts, with Joe Stoy)

Programming Research Group, Oxford

1974:

- paper: *Continuations: A Mathematical Semantics for Handling Full Jumps*
(with Chris Wadsworth, republished 2000)
- essay: *A Theory of Programming Language Semantics*
(submitted for the *Adams Prize*, with Robert Milne, 526 pp.)

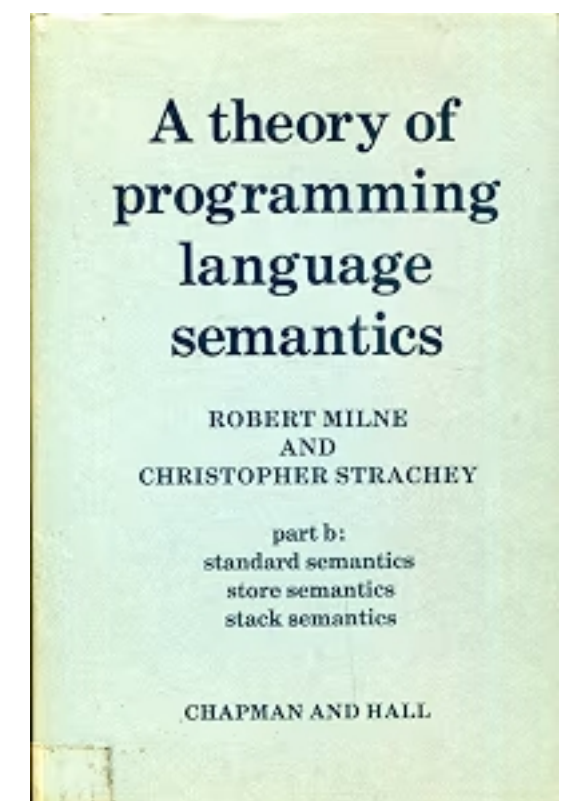
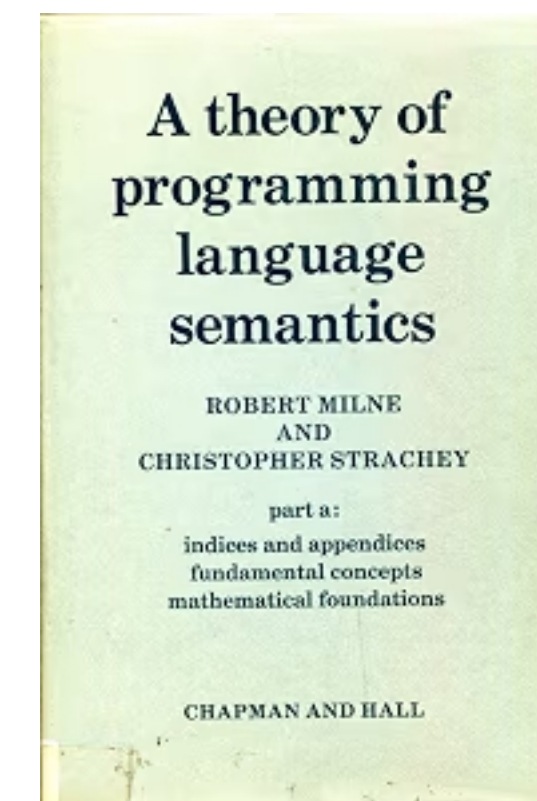
Programming Research Group, Oxford

1975:

- Strachey was diagnosed as having contracted jaundice
- at first, he appeared to recover
- then, tragically, he died – aged 58

1976:

- book: *A Theory of Programming Language Semantics*
 - Milne's *rewrite* of the *Adams Prize* essay



To conclude

Strachey, Formal Semantics, and WG 2.2



Higher-Order and Symbolic Computation, 13, 103–114, 2000
© 2000 Kluwer Academic Publishers. Manufactured in The Netherlands.

Some Reflections on Strachey and His Work

DANA SCOTT

dana_scott@cs.cmu.edu

School of Computer Science, Carnegie Mellon University, 5000 Forbes Avenue, Pittsburgh, PA 15213, USA

Christopher Strachey and I first met at an IFIP WG2.2 meeting in Vienna in the summer of 1969. This was the first one I attended. These were lively meetings with lots of arguments. I was immediately struck by Strachey's striving to isolate clear-cut general principles, and, though I did not understand everything he said, I found his approach the most sympathetic